Project - Low Level Design

on

ProjectTitle

Deploy Hospitality App to Kubernetes

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	RIYA SATHE	EN22CS301814
2	PRITAM KUMAR GHOSH	EN22CS301756
3	VINAYAK PANDIYA	EN22ME304111
4	PRANJAL DHANOTIYA	EN22CS301728
5	PREET SOLANKI	EN22CS301753

Group Name: 05D7

Project Number: DO-28

Industry Mentor Name:

University Mentor Name: Prof. Shyam Patel

Academic Year: Jan – April 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction

1.1 Scope of the document

The scope of this Low Level Design (LLD) document encompasses the AI Multi-Agent Travel Planner system. It details the internal architecture, component design, process flows, data flows, and specifics of the AI agent integration (CrewAI) responsible for generating personalized, day-wise travel itineraries. This document adheres to the system's structural constraints, including cost optimization and role delegation logic.

1.2 Intended Audience

This document is intended for:

- Software Developers and Engineers

- System Architects

- AI/ML Engineers and prompt designers

- Project Managers and Stakeholders involved in extending, scaling, or maintaining the Multi-Agent Hospitality system.

1.3 System Overview

The **AI Multi-Agent Travel Planner** is a travel itinerary generation application. Users interact via a Streamlit-based web interface, providing their desired destination, budget, and trip duration. The request is persisted into a MySQL database for tracking purposes. The core logic relies on a **CrewAI** orchestrator that spins up two robust AI agents: a "Travel Researcher" and a "Travel Itinerary Planner," powered by the **Groq LLM (`llama-3.1-8b-instant`)**. These agents work sequentially to gather data and generate a compliant, structured, day-wise itinerary that strictly adheres to the provided budget.

2. System Design

2.1 Application Design

The application follows a modular, monolithic architecture built entirely in Python.

Frontend/UI Layer: Streamlit (`app.py`), providing an interactive web-based interface with custom CSS styling.

AI Orchestration Layer: CrewAI (`agents.py`, `ai_trip_tasks.py`, `ai_trip_engine.py`), handling AI agent definitions, task assignments, and sequential workflow execution.

Persistence Layer: MySQL database (`db.py`), persisting trip requests securely.

LLM Provider: Groq API (via LangChain), specifically configured for fast inference to ensure UI responsiveness.

2.2 Process Flow

1. **User Input**: A user enters the `Destination`, `Budget`, and `Number of Days` in the Streamlit UI and clicks "Generate Trip".

2. **Validation**: The script (`app.py`) validates the input (e.g., non-empty destination).

3. **Data Persistence**: A database connection is established (`db.py`), and the user parameters are inserted into the `trips` table.

4. **AI Processing Trigger**: The orchestrator (`ai_trip_engine.py -> run_crew()`) is invoked synchronously with an active UI spinner.

5. **Agent Execution (Task 1)**: The Crew initializes. The `Travel Researcher` agent is given the parameters to gather facts, attractions, food, and transport specifics.

6. **Agent Execution (Task 2)**: The `Travel Itinerary Planner` agent consumes the Researcher's findings to draft a daily breakdown, ensuring constraints (such as daily budget division and no repeated locations) are met.

7. **Response Aggregation**: The finalized plan (in Markdown format) is returned by the Crew.

8. **UI Presentation**: Streamlit renders the resulting itinerary inside a designated result card.

2.3 Information Flow

UI → Persistence Layer: `INSERT INTO trips (destination, budget, days)`

UI → Orchestration Engine: Context parameters `(destination, days, budget)` are passed to `run_crew()`.

Engine → Tasks: Parameters dynamically populate `Task` prompt descriptions in `ai_trip_tasks.py`.

Researcher Agent ↔ Groq LLM: Submits specific queries to Langchain/Groq to fetch target locale information.

-**Researcher Agent → Writer Agent**: The output context (attractions, restaurants, etc.) is implicitly passed down the sequential CrewAI chain.

Writer Agent ↔ Groq LLM: Formats the context into strict day-wise itineraries, enforcing financial limits.

Writer Agent → Engine: The finalized text string is passed back to the Engine.

Engine → UI: Markdown string is injected into the Streamlit UI components and rendered for the user.

2.4 Components Design

2.4.1 `app.py` (Frontend UI)

- Responsible for page configuration, injecting custom CSS styles, drawing the layout (sidebar, text inputs, buttons), and main execution logic.

- Maintains UI blocks (e.g. `st.spinner`) while waiting for the AI subsystem.

- Handles interactions with the database function `get_connection()` and the engine function `run_crew()`.

2.4.2 `db.py` (Data Persistence)

- Defines the `get_connection()` function.

- Relies on `.env` variables (`MYSQL_HOST`, `MYSQL_USER`, `MYSQL_PASSWORD`, `MYSQL_DB`) to establish a MySQL connection.

2.4.3 `agents.py` (Agent Definitions)

- Configures the `ChatGroq` LLM model (`groq/llama-3.1-8b-instant`, low temperature: 0.3 for consistency).

`researcher_agent`: Assigned the role of finding landmarks and travel logistics.

`writer_agent`: Assigned the role of budget-constrained formatting and scheduling.

2.4.4 `ai_trip_tasks.py` (Task Generator)

- Dynamic task generation factories depending on real-time user arguments.

- **`research_task()`**: Focuses strictly on retrieving relevant domain data large enough to cover the `days` parameter.

- **`writer_task()`**: Extremely strict prompt structure dictating the constraints (`TOTAL TRIP BUDGET`, `DIVIDE BY DAYS`, `OUTPUT FORMAT`).

2.4.5 `ai_trip_engine.py` (Workflow Orchestrator)

- Instantiates a `Crew` object linking tasks to the agents.

- Calls `crew.kickoff()` to manage the multi-step lifecycle over the Groq LLM.

2.5 Key Design Considerations

1. **Prompt Engineering & Hard Constraints**: The prompt inside `writer_task` implements strict negative rules (e.g., "Do NOT repeat locations", "Do NOT include Google Maps links", "TOTAL trip budget, not per category") to reign in common LLM hallucination and ensure factual, constrained outputs.

2. **Delegation Restrictions**: Agents are set to `allow_delegation=False`. This design choice directs distinct execution bounds per role and prevents infinite loop delegations common in multi-agent frameworks like CrewAI.

3. **Low Latency Architecture**: To avoid application timeouts and provide a seamless web experience, the system bypasses heavy parameter models and hooks into Groq's LPU setup via LangChain for extremely rapid text generation (`llama-3.1-8b-instant`).

2.6 API Catalogue

The app utilizes internal Python integrations over public REST interfaces. Key signatures include:

- `get_connection() -> mysql.connector.connection.MySQLConnection`

- `run_crew(destination: str, days: int, budget: float) -> str`

- `research_task(destination: str, days: int) -> Task`

- `writer_task(destination: str, days: int, budget: float) -> Task`

2.7 Deployment Architecture (AWS & Docker)

The application is designed to be easily deployable using containerization.

2.7.1 Containerization with Docker

- **Dockerfile**: Defines the environment, installs OS-level dependencies, copies the application source code (`app.py`, `agents.py`, `db.py`, etc.), installs Python dependencies from `requirements.txt`, and sets the Streamlit entrypoint.

-**Docker Image**: A standalone, immutable snapshot of the application generated from the `Dockerfile`. It encapsulates the Python runtime and all application dependencies.

- **Docker Container**: The live, running instance of the Docker image. Multiple containers can be run independently for scaling purposes.

2.7.2 Docker Build & Run

The following deployment steps showcase the technical mechanics of the container setup.

1. Dockerfile Walkthrough:

- `FROM python:3.10-slim`: Uses a lightweight base Python 3.10 image to reduce the final container footprint.

- `ENV PYTHONDONTWRITEBYTECODE 1` & `ENV PYTHONUNBUFFERED 1`: Prevents Python from writing `.pyc` files to disk and forces the stdout/stderr streams to be unbuffered.

- `WORKDIR /app`: Sets the subsequent command directory to `/app`.

- `RUN apt-get update && apt-get install -y...`: Installs crucial build tools (`build-essential`) and MySQL system-level development libraries (`default-libmysqlclient-dev`) needed for standard `mysql-connector` pip installs.

- `COPY requirements.txt .`: Copies Python dependencies over first to utilize Docker's layer caching mechanism.

- `RUN pip install --no-cache-dir -r requirements.txt` : Installs python modules without storing the local cache archive to maintain a small image size.

- `COPY . .` : Imports all application code (`app.py`, `.env`, agents) into the `/app` working directory.

- `EXPOSE 8501` : Informs Docker that the container listens on the Streamlit default port `8501` at runtime.

- `CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0"]` : The default shell execution invoked when the container boots, binding Streamlit strictly to all interfaces (`0.0.0.0`) so it is reachable from outside the container networking.

2. Docker Image Build Process and Container Setup:

To deploy the application to AWS EC2, the DevOps execution follows these primary steps on the host machine where the Docker Engine is running:

1. **Build the Docker Image**:

Navigate to the directory holding the application's `Dockerfile`. Execute the `build` command, tagging `-t` the resulting image for easy reference:

```bash
docker build -t ai-travel-planner:latest .
```

**During this process, Docker executes each line from the `Dockerfile` step-by-step, downloading the base OS layer, caching unchanged dependencies, and packaging the code into the `ai-travel-planner`*

Docker Image:

```
ubuntu@ip-172-31-12-84:~/Multi-Agent-Hospitality-System$
ubuntu@ip-172-31-12-84:~/Multi-Agent-Hospitality-System$ sudo docker images
REPOSITORY          TAG         IMAGE ID       CREATED         SIZE
travel-planner-app  latest      a016b43a4955   9 minutes ago   1.61GB
python              3.10-slim   f2d628de8a5f   9 hours ago     122MB
ubuntu@ip-172-31-12-84:~/Multi-Agent-Hospitality-System$
```

2. **Setup and Run the Container**:

Initialize a new container from the created image. Map port `8501` to make the application accessible outside the Docker host using `-p`:

```bash

docker run -d --name travel-agent-app -p 8501:8501 --env-file .env ai-travel-planner:latest

```

Flags explained:

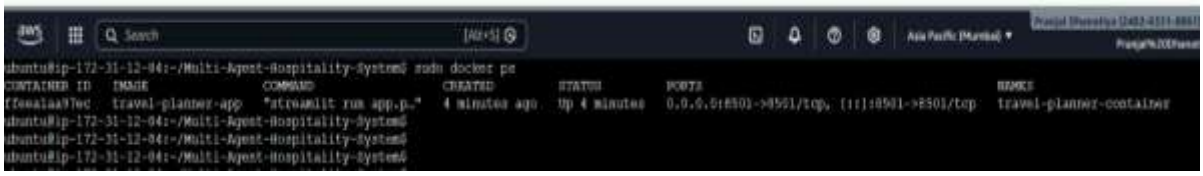
- `-d`: Runs the container in detached (background) mode.

- `--name`: Assigns a readable container name (`travel-agent-app`).

- `-p 8501:8501`: Forwards traffic targeting port 8501 on the AWS EC2 Host into port 8501 listening within the container.

- `--env-file .env`: Mounts the `.env` file directly into the container during runtime, securely injecting variables like `MYSQL_HOST` and the Groq API key without hardcoding them into the image.

Docker Container :



2.7.3 AWS Cloud Deployment (EC2)

- **Host Infrastructure**: The application is hosted on an Amazon EC2 instance (e.g., Ubuntu).

Execution Environment: The EC2 instance runs the Docker Daemon. The application's Docker Image is either built on the EC2 instance directly or pulled from a container registry (like Docker Hub or AWS ECR).

Network Configuration:

- The Streamlit application within the container exposes port `8501`.

- The Docker container binds this internal port to a host port (e.g., `8501:8501` or `80:8501`) on the EC2 instance.

- AWS Security Groups are configured to allow inbound TCP traffic on the exposed host port so that external users can access the web interface.

- The EC2 instance requires outbound internet access (via IGW or NAT Gateway) to communicate with external APIs (Groq LLM).

3. References

- CrewAI Documentation
- Streamlit Core Documentation
- Groq API Documentation
- LangChain Framework Guide
- Python MySQL Connector Docs